Porting the UberClock LiteX SoC Build to openXC7: Debugging Log, Fixes, and Status

October 11, 2026

Abstract

This note records an attempt to build the UberClock LiteX SoC (2.soc—litex/) for the Alinx AX7203 (xc7a200tfbg484—2) using the fully open-source **openXC7** flow (Yosys → nextpnr-xilinx → prjxray packer) instead of Vivado, as tracked by item 7 of the project's own roadmap ("Port from Vivado to openXC7"). It documents, in order, every error hit, its root cause, and its fix (or current workaround). A minimal configuration – RISC-V CPU, UART, integrated RAM, no DSP core, no DDR3, no Ethernet – now builds and routes to a working .bit file end to end. The full design with the UberClock DSP core enabled (−−with−uberclock) does *not* yet build cleanly: it exposes a reproducible nextpnr-xilinx placement-legality bug, independent of the earlier capacity/performance issues already fixed. Section 5 gives the current status and recommended path forward.

Contents

1 Goal and starting point **2**
1.1 Toolchain environment discovered on this machine 2

2 Stage 1: CPU + UART + integrated RAM only **2**
2.1 Issue 1 – Makefile recipe pasted into the shell instead of invoked through `make` . . . 2
2.2 Issue 2 – `PRJXRAY_DB_DIR` unset in an interactive shell 3
2.3 Issue 3 – SA placer leaves a fracturable LUT pair unbound, router then fails 3
2.4 Issue 4 – BRAM CASCADE pair placed in non-adjacent columns 4
2.5 Issue 5 – `fasm2frames` not found 4
2.6 Stage 1 result: success 5

3 Stage 2: enabling the UberClock DSP core **5**
3.1 Issue 6 – `-flatten` before optimization turns a multi-channel design into a multi-hour synthesis run 5
3.2 Issue 7 – without abc9, the mapped netlist is 117% over LUT capacity 6
3.3 Issue 8 – HeAP placer cannot legalize an abc9-generated CARRY4 chain (reproducible, not seed luck) 7
3.4 Issue 9 – disabling CARRY4 does not fix the bug, it just relocates it, non-deterministically 8
3.5 Issue 10 – SA placer with abc9: completes annealing, then hits a third, independent nextpnr-xilinx legalization bug 9

4 Summary of attempts **10**

5 Current status and recommended path forward **10**
5.1 What works today 10
5.2 What does not work today 10
5.3 Recommended next steps, in order of effort 11

A Makefile changes **12**

B `soc.py` changes **12**

C `setup_dev.sh` changes **13**

1 Goal and starting point

The project's existing Vivado-based LiteX build flow (2.soc—litex/3.build/Makefile, target script 8.python/src/targets/alinx_ax7203_uberclock.py, SoC assembly in 8.python/src/uberclock_soc/soc.py) already supports multiple toolchains through LiteX's XilinxPlatform abstraction. For 7-series devices LiteX lists four supported toolchains: vivado, f4pga, yosys+nextpnr, and openxc7 (the latter two both route through XilinxYosysNextpnrToolchain, differing only in where chip databases and the prjxray device database are expected to live). No RTL changes were assumed to be necessary going in; the question was whether the existing design would synthesize, place, and route on the open flow at all, and if not, why.

1.1 Toolchain environment discovered on this machine

openXC7 was already installed, but not where LiteX's toolchain wrapper defaults expect it (a snap path). The actual layout:

```
/opt/openxc7/bin/ nextpnr-xilinx, yosys, bbasm, xc7frames2bit,
                          xc7patch, bitread, bittool, segmatch,
                          frame_address_decoder, gen_part_base_yaml
/opt/openxc7/share/prjxray-db/ device database (artix7/xc7a200t*)
/opt/openxc7/share/nextpnr-xilinx/python/ bbaexport.py (chipdb generator)
/opt/openxc7/setup_env.sh ready-made env-var setup script
~/openxc7-chipdb/xc7a200tfbg484.bin pre-generated chip database (317 MB)
```

The pre-generated chip database's filename was confirmed to match exactly what LiteX's toolchain computes for this part. The toolchain derives a dbpart string from the device name via the regex xc7([aksz])([0—9]+)(.∗)—([0—9]) applied to xc7a200tfbg484—2, giving xc7a200tfbg484 – an exact match to the cached file, so chip database generation (otherwise a slow, memory-hungry step run through bbaexport.py + bbasm) was skipped on every build in this session.

2 Stage 1: CPU + UART + integrated RAM only

The first goal was the smallest possible configuration: RISC-V CPU, UART, and the default integrated (BRAM-based) RAM, with no UberClock DSP core, no UberDDR3, no Ethernet, no HDMI.

2.1 Issue 1 – Makefile recipe pasted into the shell instead of invoked through `make`

The first attempt failed before even reaching the toolchain:

```
make build-board:
    mkdir -p $(CHIPDB)
    ...
make: *** No rule to make target 'build-board:'. Stop.
zsh: command not found: CHIPDB
mkdir: missing operand
```

Cause. The Makefile recipe text had only been *described* in a prior chat turn, not actually written to the Makefile. It was pasted directly into the interactive zsh prompt. zsh then (a) tried to run make build−board: literally, with a trailing colon, which is not a valid target name, and (b) treated $(CHIPDB) as shell command substitution – i.e. "run a command literally named CHIPDB and substitute its output" – which obviously doesn't exist as a command.

Fix. The toolchain-aware recipe was actually added to 2.soc−litex/3.build/Makefile (see Appendix A) and invoked the correct way:

```
make build-board TOOLCHAIN=openxc7 OPTIONS='...'
```

2.2 Issue 2 – PRJXRAY_DB_DIR unset in an interactive shell

A manual sanity check also failed:

```
$ ls $PRJXRAY_DB_DIR/artix7/ | grep a200t
ls: cannot access '/artix7/': No such file or directory
```

Cause. PRJXRAY_DB_DIR (and CHIPDB, NEXTPNR_XILINX_PYTHON_DIR) only existed inside the Makefile recipe, never exported into the interactive shell.

Fix. Two options, both now valid: run source /opt/openxc7/setup_env.sh (a ready-made script already present on disk), or simply go through make build−boardTOOLCHAIN=openxc7, which now exports these variables itself (see Appendix A).

2.3 Issue 3 – SA placer leaves a fracturable LUT pair unbound, router then fails

With the Makefile fixed and TOOLCHAIN=openxc7 building correctly, the first real toolchain failure appeared during place-and-route of the CPU-only design:

```
Warning: fixupPlacement: slot with 7 unique inputs (>6): lut5=...
  lut6=...; skip
Warning: fixupPlacement: slot with 8 unique inputs (>6): lut5=...
  lut6=...; skip
  ... (150 such warnings total)
Info: routing clock 's7mmcm0_clkout_buf'
Info: failed to find a route using dedicated resources.
Info: Running router2...
ERROR: No wire found for port O6 on source cell
  $abc$27999$auto$blifparse.cc:536:parse_blif$28010.
150 warnings, 1 error
```

Cause. LiteX's XilinxYosysNextpnrToolchain hardcodes −−placer sa with no CLI override (litex/build/xilinx/yosys_nextpnr.py, constructor). On this LUT-dense, fracturable-LUT-packed design (VexRiscv + its decode/ALU logic), nextpnr-xilinx's SA placer runs a post-placement fixupPlacement pass that tries to legalize LUT5/LUT6 pairs sharing one slice's six input pins. When it can't legalize a given pair it just skips – but this can leave one of the two cells unbound to any BEL. The router then has no wire assignment for that cell's output and fails.

Fix. Patched uberclock_soc/soc.py to override the placer right after platform construction:

```
platform = alinx_ax7203.Platform(toolchain=toolchain)

if toolchain in ("openxc7", "yosys+nextpnr"):
    platform.toolchain._pnr_opts = platform.toolchain._pnr_opts.replace(
        "--placer sa", "--placer heap"
    )
```

Confirmed heap is a supported placer (nextpnr−xilinx −−help lists sa, heap; the binary's own undocumented default is actually heap, but LiteX's wrapper always passes sa explicitly, overriding that default). The heap placer does not hit this particular legalization bug.

2.4 Issue 4 – BRAM CASCADE pair placed in non-adjacent columns

With the placer fixed, a *different* routing failure appeared:

```
ERROR: Failed to route arc 0 of net 'main_ram.0.8.genblk1.genblk1.CAS_B',
  from SITEWIRE/RAMB36_X1Y35/CASCADEOUTB to SITEWIRE/RAMB36_X0Y45/CASCADEINB.
```

Cause. The design's default integrated RAM is 256 KiB (INTEGRATED_MAIN_RAM_BYTES $= 256 * 1024$ in BaseSoC). At that depth, Yosys's BRAM inference uses a depth-cascaded RAMB36E1 pair (dedicated CASCADEOUT/ CASCADEIN wiring, same-column-only resources on 7-series). nextpnr-xilinx placed the two halves in *different, non-adjacent columns* (RAMB36_X1Y35 and RAMB36_X0Y45) – a placement that is physically unroutable, because it is not enforcing the cascade pair's required column adjacency as a relative placement constraint.

Fix (workaround, not a real fix). Reduced the integrated RAM to 64 KiB for openXC7 builds via the already-exposed LiteX CLI flag:

```
--integrated-main-ram-size=0x10000
```

small enough that Yosys no longer needs BRAM cascading for this memory. **The 256 KiB default remains broken under openXC7.** A real fix needs either an upstream nextpnr-xilinx fix to enforce CASCADE-pair adjacency during placement, or a LiteX-side change to compose the RAM differently (e.g. parallel narrower blocks addressed by higher bits) for this toolchain.

2.5 Issue 5 – `fasm2frames` not found

The final blocker for Stage 1:

```
build_alinx_ax7203.sh: line 5: fasm2frames: command not found
```

Cause. The generated build script's packer stage is:

```
fasm2frames --part xc7a200tfbg484-2 --db-root .../prjxray-db/artix7 \
  alinx_ax7203.fasm > alinx_ax7203.frames
xc7frames2bit --part_file .../part.yaml --part_name xc7a200tfbg484-2 \
  --frm_file alinx_ax7203.frames --output_file alinx_ax7203.bit
```

fasm2frames is *not* the fasm PyPI package (pip install fasm only provides the fasm text-format CLI, a different tool, and was tried first on this mistaken assumption – it installed fine but did not provide fasm2frames). The real fasm2frames is a Python script, utils/fasm2frames.py, that lives in the `f4pga/prjxray` repository itself, exposed as a console script via that repo's setup.py:

```
entry_points={
    'console_scripts': ['fasm2frames=utils.fasm2frames:main'],
}
```

It is not published to PyPI under this name, and this machine's /opt/openxc7 binary install does not ship it either.

Fix. Sparse-cloned just the needed subdirectories, fixed an upstream packaging omission (utils was never listed in packages =[...] , even though the entry point depends on it), and installed it editable into the LiteX venv:

```
git clone --depth 1 --filter=blob:none --sparse \
  https://github.com/f4pga/prjxray.git prjxray_src
cd prjxray_src && git sparse-checkout set utils prjxray
sed -i "s/packages=\['prjxray'\]/packages=['prjxray', 'utils']/" setup.py
pip install -e .
```

This also pulled in the real fasm, intervaltree , pyjson5, pyyaml, and simplejson dependencies declared in prjxray's setup.py. For reproducibility on a fresh machine, the same steps were added to 2.soc−litex/setup_dev.sh behind a new −−with−openxc7 flag (Appendix C).

2.6 Stage 1 result: success

```
$ make build-board TOOLCHAIN=openxc7 \
    OPTIONS='--integrated-main-ram-size=0x10000 --nextpnr-seed=42'
...
exit=0

$ file alinx_ax7203.bit
alinx_ax7203.bit: Xilinx BIT data - from alinx_ax7203.frames;
  Generator=xc7frames2bit - for xc7a200tfbg484-2
  - built 2026/10/11(18:21:09) - data length 0x947a5c
```

A complete, working bitstream for RISC-V CPU + UART + 64 KiB integrated RAM, built entirely with Yosys + nextpnr-xilinx + prjxray, no Vivado involved.

3 Stage 2: enabling the UberClock DSP core

The next step added −−with−uberclock: the 11-channel DSP pipeline (6 rx channels including a reference channel, 5 tx channels), each instantiating a CORDIC stage chain plus CIC and half-band (HB) decimation/interpolation filters (1.hw/uberclock/, 1.hw/cordic/, 1.hw/ filters /). DDR3 and Ethernet remained disabled throughout Stage 2 – those are expected to need Vivado regardless (dropped INTERNAL_VREF constraint, IDELAY/SERDES- heavy PHYs, no real static timing analysis under this flow – see Section 5).

3.1 Issue 6 – `-flatten` before optimization turns a multi-channel design into a multi-hour synthesis run

The very first −−with−uberclock attempt never finished. After nearly 10 minutes it was still deep inside Yosys's optimization fixpoint loop, past its 26th round, with no error:

```
26.14.31. Executing OPT_MUXTREE pass (detect dead branches in mux trees).
  ... <suppressed ~5669 debug messages>
26.14.32. Executing OPT_REDUCE pass (consolidate $*mux and $reduce_* inputs).
make: *** [Makefile:63: build-board] Terminated
```

Cause. LiteX's default synth options are "−flatten −nosrl −arch xc7 ". Yosys's own synth_xilinx documentation (yosys −p "helpsynth$_x ilinx$")$confirms$−flatten$is applied in the$**prepare:**$stage$−−$*i.e. before*map_ds

$runsitsentire(expensive, super - linearinplaces)optimizationpipelineonthefull$~11×-larger flat netlist, instead of optimizing each unique module once and simply copying the result 11 times at the end.

Fix. YosysNextPNRToolchain exposes a documented, first-class extension point for exactly this, self._yosys_template (default [], meaning "use the built-in default template"). Patched soc.py to strip –flatten from _synth_opts and supply a custom template that keeps hierarchy through all of synth_xilinx's internal optimization, flattening only immediately before the write_json step that nextpnr-xilinx's flat-netlist JSON input requires:

```
platform.toolchain._synth_opts = platform.toolchain._synth_opts.replace(
    "-flatten ", ""
)
platform.toolchain._yosys_template = [
    "verilog_defaults -push",
    "verilog_defaults -add -defer",
    "{read_files}",
    "verilog_defaults -pop",
    "attrmap -tocase keep -imap keep=\"true\" keep=1"
      " -imap keep=\"false\" keep=0 -remove keep=0",
    "{yosys_cmds}",
    "synth_{target} {synth_opts} -top {build_name}",
    "flatten",
    "clean",
    "write_{write_fmt} {write_opts} {output_name}.{synth_fmt}",
]
```

With this change, synthesis for the full 11-channel design actually completes (where it previously did not, even given 30 minutes).

3.2 Issue 7 – without abc9, the mapped netlist is 117% over LUT capacity

With the flatten fix alone (placer heap, default Yosys abc mapper, 64 KiB RAM), synthesis completed and placement began, then failed:

```
Info: Creating initial analytic placement for 191206 cells, ...
ERROR: Failed to expand region (0, 0) |_> (264, 260) of 316493 SLICE_LUTXs
```

and nextpnr's own pre-placement utilization report showed:

```
Info: Created 314717 SLICE_LUTX cells from:
Info: 77172x FDRE
Info:  SLICE_LUTX: 316493/269200 117%
```

Cause (confirmed, not assumed). To rule out a real DSP-inference regression from the Issue 6 fix, this was cross-checked against the project's own *known-working* Vivado synthesis report (alinx_ax7203_utilization_synth .rpt) for a config that additionally includes the full UberDDR3 and Ethernet logic (i.e. a *larger* design than this openXC7 attempt):

Resource	Vivado (DDR3+Eth+UberClock)	This run (UberClock only)
DSP48E1	168	168 (exact match)
RAMB36E1	290	179
RAMB18E1	22	19
Slice LUTs / SLICE_LUTX	74448 / 134600 (55.3%)	316493 / 269200 (117%)

Table 1: Vivado vs. this run's Yosys (`abc`, no `-abc9`) synthesis, same UberClock RTL. (nextpnr-xilinx's `SLICE_LUTX` capacity is reported on a 2× finer basis than Vivado's whole-LUT6 `Slice LUTs` count, so the two denominators differ by design, not by a counting bug.)

The DSP48E1 count matching exactly (168 = 168) confirms DSP inference itself was not the regression. The LUT blowup is specific to Yosys's classic (non-abc9) abc LUT technology mapper, which is well known to be markedly less efficient than Vivado's proprietary mapper on this kind of RTL (deep CORDIC pipeline stages, case-driven multiplexing).

Fix. Added ––yosys–abc9 (already exposed on LiteX's CLI via xilinx_yosys_nextpnr_args, no code change required) to switch to Yosys's newer abc9 LUT mapper. Result: total cell count dropped from 191,206 to 131,503, and SLICE_LUTX: 197705/269200 73% – comfortably within capacity (DSP48E1 still exactly 168; RAMB36E1 179/365 = 49%; RAMB18E1 19/730 = 2%).

3.3 Issue 8 – HeAP placer cannot legalize an abc9-generated CARRY4 chain (reproducible, not seed luck)

With abc9 added, placement progressed much further, then failed differently:

```
Info: Running main analytical placer.
ERROR: Unable to find legal placement for cell
  '$flatten\uberclock.\cordic_nco.\cordic_stages[16].stage_inst.
   $techmap$auto$opt_share.cc:199:merge_operators$2158173.
   $auto$alumacc.cc:512:replace_alu$2429962.genblk1.slice[0].genblk1.carry4',
  check constraints and utilisation.
```

At the time of this failure, overall utilization was comfortably within budget (SLICE_LUTX 73%, SLICE_FFX 39%, CARRY4 48%, RAMB36E1 49%, DSP48E1 22%) – ruling out a capacity problem. This is a genuine **placement-legality bug**: nextpnr-xilinx's HeAP placer's legalizer cannot place this specific CARRY4 chain shape, which comes from Yosys abc9's alumacc/replace_alu ALU-extraction pass as applied to the CORDIC pipeline's adders.

To test whether this was placement-seed bad luck (as Issue 4's cascade bug had been), the build was repeated with a different ––nextpnr–seed (7 instead of 42), everything else identical. Result: the *same error signature*, at the *same point* in placement (immediately on entering "Running main analytical placer"), but on a *different* CARRY4 instance (cordic_cpu5. cordic_stages [14] instead of cordic_nco. cordic_stages [16]):

```
ERROR: Unable to find legal placement for cell
  '$flatten\uberclock.\cordic_cpu5.\cordic_stages[14].stage_inst...
   genblk1.carry4', check constraints and utilisation.
```

Two different seeds hitting the identical failure mode on two different (but structurally identical) instances indicates the bug is **systematic** – HeAP cannot legalize *any* CARRY4 of this abc9-generated shape in this design – not a probabilistic, seed-dependent placement failure the way Issue 4 was. Further seed rolling was judged unlikely to help and was not pursued further.

3.4 Issue 9 – disabling CARRY4 does not fix the bug, it just relocates it, non-deterministically

As a workaround for Issue 3.3, –nocarry was added to synth_xilinx's options (forcing the CORDIC adders into plain LUT logic instead of dedicated CARRY4 chains), keeping abc9 and the heap placer:

```
if os.environ.get("OPENXC7_NOCARRY"):
    platform.toolchain._synth_opts += "-nocarry "
```

First attempt (30-minute cap). The CARRY4 crash disappeared (CARRY4: 0/33650 0%), LUT utilization remained well within budget (SLICE_LUTX: 152384/269200 56%), total placer cell count grew to 204,521 (from 131,503 with CARRY4 enabled), and the HeAP placer printed *no* iteration output at all before the 30-minute cap was hit:

```
Info: Running main analytical placer.
make: *** [Makefile:63: build-board] Terminated
```

Taken alone, this looked like non-convergence rather than a crash.

Second attempt, same command, no premature cap. To find out whether that was genuine non-convergence or just a slow instance of the same failure, the identical command was re-run with no artificial time limit (a 4-hour safety net instead of a 30-minute cutoff). This time it did *not* hang: it reached the identical point – "`Running main analytical placer.`" – and failed on the very next line, almost instantly, with the same class of error as Issue 8 but on a *different* cell, this time one produced by Yosys abc9's own internal reintegration pass rather than the CORDIC adder's CARRY4:

```
Info: Running main analytical placer.
ERROR: Unable to find legal placement for cell
  '$flatten\uberclock.$auto$abc9_ops.cc:1552:reintegrate$3585864',
  check constraints and utilisation.
```

As before, utilization at the point of failure was comfortably within budget (SLICE_LUTX: 152441/269200 56%, CARRY4:0/33650 0*zero* "at iteration #1..." lines were ever printed; the failure happens before annealing proper even starts.

Revised conclusion. Two runs of the *exact same command* (same seed, same options) produced different symptoms – one an apparent 30-minute hang with no output, the other an almost-instant crash – at the identical point in placement. –nocarry does not fix Issue 3.3's bug; it relocates the same underlying nextpnr-xilinx placer weakness (failing to legalize a specific abc9-produced cell during HeAP's very first legalization step, before any annealing) from a CARRY4 chain onto a different cell type, and whether that manifests as a quick, explicit error or an unbounded-looking stall varies non-deterministically between otherwise identical runs. This is a stronger finding than "disabling CARRY4 regresses convergence": the bug is not specific to CARRY4 at all, and no synthesis-option change tried so far removes it, only moves it.

Conclusion at this point. CARRY4 enabled ⇒ placer fails to legalize a CARRY4 cell (Issue 8). CARRY4 disabled ⇒ placer fails to legalize a different abc9-internal cell instead, non-deterministically presenting as either a crash or an apparent hang (Issue 9). Both paths hit the same class of nextpnr-xilinx placer bug; neither is a real fix.

3.5 Issue 10 – SA placer with abc9: completes annealing, then hits a third, independent nextpnr-xilinx legalization bug

Keeping abc9 and CARRY4 enabled (i.e. without −−nocarry), the placer was switched back to sa – a combination not yet tried, since the original sa failure (Issue 3) was observed on a much smaller, non-abc9 CPU-only netlist with a completely different cell mix:

```
OPENXC7_PLACER=sa \
make build-board TOOLCHAIN=openxc7 \
  OPTIONS='--with-uberclock --integrated-main-ram-size=0x10000 \
          --nextpnr-seed=42 --yosys-abc9'
```

(enabled via a new, temporary OPENXC7_PLACER environment-variable override in soc.py, see Appendix B).

First attempt (30-minute cap). No crash and no placement-legality error were observed, but the run was only ∼19% through nextpnr's very first, non-optimizing placement pass when the cap was hit (see the original draft of this section for the exact trace) – inconclusive on whether sa would eventually also hit Issue 8's bug class.

Second attempt, no premature cap (6-hour safety net). This time the run went much further: it completed sa's entire annealing schedule – 272 iterations, converging on a stable timing cost, over 5527 seconds (≈92 minutes) of actual SA placement time – and only then failed, during post-placement legalization:

```
Info: at iteration #272: temp = 0.000000, timing cost = 11993, wirelen = 71860986
Info: SA placement time 5527.12s
ERROR: post-placement validity check failed for Bel 'SLICE_X0Y249/A5FF' (no cell)
ERROR: Placing design failed.
```

Utilization at the point of failure was identical to Issue 8's heap run on the same netlist (SLICE_LUTX 197707/269200 = 73%, CARRY4 16209/33650 = 48%) – again not a capacity problem. nextpnr's own timing analysis also flagged 64 combinational loops in the netlist throughout the run, explicitly attributed to abc9 reintegration and treated as non-fatal by default; this did not cause the failure but is noted here as a property of the abc9-mapped netlist worth tracking if timing closure is attempted later.

This is a third, independently reproducible bug signature. The exact message "post−placement validity (no cell)" was also seen, on a *different, earlier* netlist, during Stage 1 seed-rolling: the very first attempt to use the heap placer on the (pre-abc9) 256 KiB-RAM CPU-only build hit "SLICE_X10Y181/A5FF" in exactly the same way, when heap's own brief internal SA-refinement sub-step ran after its analytic placement. That occurrence was resolved along the way to Stage 1's working configuration (Section 2) and was not written up as its own numbered issue there; it is referenced here because the same bug reappearing, independently, on a completely different (larger, abc9-mapped) netlist confirms it is a general nextpnr-xilinx SA-legalization weakness, not an artifact specific to one design size or one synthesis configuration.

Conclusion. sa does not avoid Issue 8's problem – it runs substantially longer (a full, successful anneal) and then hits a *different* systematic nextpnr-xilinx legalization bug instead. Across Issues 8–10, every placer/synthesis-option combination tried (heap; heap+−−nocarry; sa) on the full UberClock netlist has hit an independent placement-legality bug, and one of those bug signatures (A5FF (no cell)) recurs across two unrelated netlist configurations. This is strong evidence of a genuine, multi-faceted toolchain maturity gap rather than a single fixable misconfiguration on this project's side.

4 Summary of attempts

Configuration	Key toolchain options	Result	Failure / outcome
CPU+UART+RAM	sa placer (LiteX default)	Fail	LUT5/6 fixupPlacement → unroutable (Issue 3)
CPU+UART+RAM	heap placer	Fail	BRAM CASCADE non-adjacent (Issue 4)
CPU+UART+64KiB RAM	heap placer	**Success**	Working .bit
+UberClock	heap, −flatten (default)	Fail (no error)	Non-converging synth, 10+ min, round 26 of opt loop (Issue 6)
+UberClock	heap, hierarchy-preserving flatten	Fail	117% LUT overcapacity (Issue 7)
+UberClock	heap, +abc9	Fail	CARRY4 placement-legality bug, seed 42 (Issue 8)
+UberClock	heap, +abc9, seed 7	Fail	Same bug class, different CARRY4 instance (Issue 8)
+UberClock	heap, +abc9, +−nocarry	Fail (2×)	Same bug class on a different cell; hang vs. instant crash, non-deterministic (Issue 9)
+UberClock	sa, +abc9	Fail	Full 92-min anneal converges, then `A5FF (no cell)` legalization bug – 3rd distinct bug signature (Issue 10)

Table 2: All configurations attempted, in chronological order.

5 Current status and recommended path forward

5.1 What works today

A complete, Vivado-free build for RISC-V CPU + UART + 64 KiB integrated RAM on the Alinx AX7203, via:

```
make build-board TOOLCHAIN=openxc7 \
  OPTIONS='--integrated-main-ram-size=0x10000 --nextpnr-seed=42'
```

This exercises the full open flow – Yosys, nextpnr-xilinx, prjxray's fasm2frames and xc7frames2bit – end to end, and is loadable via the existing OpenFPGALoader-based programmer (alinx_ax7203.py's create_programmer, unchanged by this work).

5.2 What does not work today

−−with−uberclock (the DSP core) does not yet build cleanly under openXC7. The blocking issue is *not* capacity – it is nextpnr-xilinx placement-legalization bugs, and testing has now turned up **three independent ones** on this one design:

- heap fails to legalize a specific abc9- produced cell during its very first legalization step, before any annealing runs – first seen on a CARRY4 chain from the CORDIC adders (Section 3.3), reproducible across random seeds.
- Disabling CARRY4 inference (Issue 9) does not fix that bug, it relocates it onto a different abc9-internal cell, presenting non-deterministically as either an instant error or an apparent multi-hour stall across otherwise-identical runs.
- Switching to the sa placer (Issue 10) avoids both of the above – it completes a full, successful 92-minute anneal – but then hits a *third*, independent bug during post-placement legalization ("post−placement validity check failed for Bel'¡slice¿/A5FF' (no cell)"), the same exact signature already seen once before, on an unrelated netlist, during Stage 1 seed-rolling.

No placer/synthesis-option combination tried gets past placement for the full design, and one bug signature (A5FF (no cell)) has now been independently reproduced on two different netlists, which makes it the single strongest upstream bug-report candidate of the three.

−−with−uberddr3, −−with−ethernet, and −−with−hdmi were not attempted under openXC7 in this session and are expected to need Vivado regardless, independent of the issues above:

- UberDDR3's INTERNAL_VREF and BITSTREAM.STARTUP.MATCH_CYCLE Vivado properties are silently dropped by XilinxPlatform.add_platform_command for this toolchain (with a printed warning) – without INTERNAL_VREF, the DDR3 I/O bank has no reference level.
- The DDR3 PHY instantiates 27×OSERDESE2, 12×ISERDESE2, 9×IDELAYCTRL, 13×IDELAYE2, in VAR_LOAD calibration mode – the least-exercised part of nextpnr-xilinx's primitive support.
- add_false_path_constraint is a documented no-op in LiteX's XilinxYosysNextpnrToolchain; nextpnr-xilinx derives only a single global −−freq from the highest create_clock, with no real propagation through the MMCM – so the DDR3 PHY's 400 MHz domain is effectively unconstrained and unverified under this flow regardless of the CARRY4 issue.

5.3 Recommended next steps, in order of effort

1. **Treat all three legalization bugs (Issues 8–10) as genuine upstream nextpnr-xilinx limitations**, not something fixable from this project's RTL or LiteX integration. Every placer/option combination tried hit one: heap hits the abc9- cell bug directly; −nocarry relocates it; sa avoids both but hits the independent A5FF (no cell) bug after a full, successful anneal. Prioritize filing the A5FF (no cell) bug first – it is the one already reproduced independently on two unrelated netlists (Stage 1's pre-abc9 CPU-only build and Stage 2's full abc9-mapped design), the strongest signal of a general, fixable toolchain defect rather than a one-off. File minimal reproducers (a single CORDIC stage module is likely enough for the abc9-cell bug) against openXC7 or nextpnr-xilinx upstream, and check whether a newer toolchain build already fixes either.

2. **In the meantime**, keep Vivado as the synthesis path for any configuration that includes −−with−uberclock, −−with−uberddr3, −−with−ethernet, or −−with−hdmi. Use openXC7 for the CPU-only bring-up configuration documented above, and for any future peripheral-free RTL bring-up work where a fully open flow is valuable.

3. **Longer term, if pursuing openXC7 further**: consider whether avoiding abc9's alumacc/replace_alu ALU-extraction specifically (rather than disabling CARRY4 globally with −nocarry, which regressed placer convergence) is possible – e.g. by restructuring the CORDIC stage's adders in RTL to a shape Yosys maps more conventionally, or by scripting a custom Yosys

pass ordering that skips just that substitution. This is invasive and was not attempted in this session.

Appendix

A Makefile changes

Added to 2.soc–litex/3.build/Makefile:

```
# Toolchain: vivado (default) or openxc7
TOOLCHAIN ?= vivado

# openXC7 install (actual layout on this machine: /opt/openxc7, not snap)
OPENXC7_DIR ?= /opt/openxc7
CHIPDB ?= $(HOME)/openxc7-chipdb
PRJXRAY_DB_DIR ?= $(OPENXC7_DIR)/share/prjxray-db
NEXTPNR_XILINX_PYTHON_DIR ?= $(OPENXC7_DIR)/share/nextpnr-xilinx/python
```

and the build–board recipe:

```
build-board:
	@echo "[build] target=$(TARGET_PY)"
	@echo "[build] toolchain=$(TOOLCHAIN) sys_clk=$(FREQ) options=$(OPTIONS)"
	source $(LITEX_VENV)/bin/activate && \
	if [ "$(TOOLCHAIN)" = "openxc7" ]; then \
	  mkdir -p "$(CHIPDB)"; \
	  export PATH="$(OPENXC7_DIR)/bin:$$PATH"; \
	  export CHIPDB="$(CHIPDB)"; \
	  export PRJXRAY_DB_DIR="$(PRJXRAY_DB_DIR)"; \
	  export NEXTPNR_XILINX_PYTHON_DIR="$(NEXTPNR_XILINX_PYTHON_DIR)"; \
	fi; \
	PYTHONPATH=$(PYTHONPATH_REPO) \
	$(PYTHON) $(TARGET_PY) \
	  --build \
	  --toolchain=$(TOOLCHAIN) \
	  --sys-clk-freq=$(FREQ) \
	  $(OPTIONS)
```

B `soc.py` changes

Added to 8.python/src/uberclock_soc/soc.py, immediately after platform construction in BaseSoC.__init__:

```
platform = alinx_ax7203.Platform(toolchain=toolchain)

if toolchain in ("openxc7", "yosys+nextpnr"):
    # Issue 3 / Issue 10: placer choice, overridable while bisecting
    # nextpnr-xilinx placement-legality bugs (sa: fracturable-LUT-pair
    # fixupPlacement bug; heap: CARRY4 legalization bug on abc9's
    # alumacc/replace_alu shapes). TEMPORARY env-gated override.
    _placer = os.environ.get("OPENXC7_PLACER", "heap")
    platform.toolchain._pnr_opts = platform.toolchain._pnr_opts.replace(
        "--placer sa", f"--placer {_placer}"
    )

    # Issue 6: keep hierarchy through synth_xilinx (don't -flatten before
    # optimization), flatten only right before nextpnr's required-flat
```

```
    # JSON output.
    platform.toolchain._synth_opts = platform.toolchain._synth_opts.replace(
        "-flatten ", ""
    )
    platform.toolchain._yosys_template = [
        "verilog_defaults -push",
        "verilog_defaults -add -defer",
        "{read_files}",
        "verilog_defaults -pop",
        "attrmap -tocase keep -imap keep=\"true\" keep=1"
          " -imap keep=\"false\" keep=0 -remove keep=0",
        "{yosys_cmds}",
        "synth_{target} {synth_opts} -top {build_name}",
        "flatten",
        "clean",
        "write_{write_fmt} {write_opts} {output_name}.{synth_fmt}",
    ]

    # Issue 9: TEMPORARY env-gated experiment, disables CARRY4 inference
    # to sidestep Issue 8's placement-legality bug. Regressed placer
    # convergence instead (see Issue 9) -- not enabled by default.
    if os.environ.get("OPENXC7_NOCARRY"):
        platform.toolchain._synth_opts += "-nocarry "
```

Note: the OPENXC7_PLACER and OPENXC7_NOCARRY environment-variable gates are deliberately temporary, added to bisect the Stage 2 failures without committing to a default before knowing which combination (if any) actually works. Once Issue 10 resolves, these should be collapsed into either a fixed default or a proper ––yosys–nocarry-style CLI flag, not left as undocumented environment variables.

C `setup_dev.sh` changes

Added a ––with–openxc7 flag to 2.soc–litex/setup_dev.sh that performs the same fasm2frames install as Issue 5's manual fix, so a fresh machine doesn't have to rediscover it:

```
if [[ "${WITH_OPENXC7}" -eq 1 ]]; then
  echo "--- Installing fasm2frames (prjxray) for openXC7 ---"
  PRJXRAY_SRC_DIR="${LITEX_DIR}/prjxray_fasm2frames"
  if ! python -c "import utils.fasm2frames" >/dev/null 2>&1; then
    rm -rf "${PRJXRAY_SRC_DIR}"
    git clone --depth 1 --filter=blob:none --sparse \
      https://github.com/f4pga/prjxray.git "${PRJXRAY_SRC_DIR}"
    (cd "${PRJXRAY_SRC_DIR}" && git sparse-checkout set utils prjxray)
    sed -i "s/packages=\['prjxray'\]/packages=['prjxray', 'utils']/" \
      "${PRJXRAY_SRC_DIR}/setup.py"
    pip install -e "${PRJXRAY_SRC_DIR}"
  fi
  command -v fasm2frames >/dev/null 2>&1 \
    && echo "fasm2frames OK: $(command -v fasm2frames)" \
    || echo "WARNING: fasm2frames still not on PATH after install."
fi
```